

Foundations of a Visualization Tool for NASA Gateway and Lunar Surface Operations

Chawin Mingsuwan

Computer Graphics Technology Aeronautical and Astronautical Engineering
Purdue University
West Lafayette, USA
cmingsuw@purdue.edu

Louis Desano

Aeronautical and Astronautical Engineering
Purdue University
West Lafayette, USA
ldesano@purdue.edu

Brasen Garcia

Aeronautical and Astronautical Engineering
Purdue University
West Lafayette, USA
garci911@purdue.edu

Jai Agarwal

Mechanical Engineering
Purdue University
West Lafayette, USA
agarw390@purdue.edu

John Ong

Computer Science
Purdue University
West Lafayette, USA
ong52@purdue.edu

Adriana Sanchez

Biological Engineering
Purdue University
West Lafayette, USA
sanch328@purdue.edu

Christine Xu

Computer Science and Artificial Intelligence
Purdue University
West Lafayette, USA
xu1906@purdue.edu

Sawyer Cawthon

Computer Science
Angelo State University
San Angelo, USA
scawthon@angelo.edu

Cung Lian

Computer Science
Angelo State University
San Angelo, USA
clian@angelo.edu

Adarsh Prathap

Mechanical Engineering
Purdue University
West Lafayette, USA
aprathap@purdue.edu

Cora Wood

Computer Engineering
Purdue University
West Lafayette, USA
wood297@purdue.edu

Haleigh Nyberg

Earth, Atmospheric, and Planetary Sciences
Purdue University
West Lafayette, USA
haleigh@purdue.edu

Abstract—The interconnected nature of the NASA Artemis missions requires immersive modeling to effectively visualize and address critical planning challenges. To lay the foundations for such modeling, Barrios Technology partnered with an interdisciplinary student team from Purdue University's Data Mine and Angelo State University to develop the Gateway Visualization Tool (GVT)—an interactive, extended reality (XR) platform built in Unreal Engine. Designed to facilitate digital engineering workflows, mission planning, and astronaut training, GVT users can manipulate Gateway's modules, rotate solar panels and radiators, change camera perspectives, and experience XR integration. Through these interactive features, GVT offers early insight into a simulated Gateway Space Station environment, providing foundational understanding of how such visualization tools can benefit the Artemis missions. This paper outlines key engineering and user experience decisions integral to GVT and outlines future development paths, including VR functionality and Lunar Surface Operations.

Index Terms—Gateway, Extended Reality, Virtual Reality, Astronaut Training, Simulation, Artemis, Mission Planning

I. INTRODUCTION

The NASA Artemis program aims to establish a sustained human presence on the lunar surface [1], marking a significant milestone in space exploration. Central to this effort is the Gateway—a modular space station that will orbit the Moon and function as a communications hub, science laboratory, and

staging point for lunar surface operations [2]. As with all components of the Artemis program, the Gateway's multifaceted nature and goal of supporting lunar surface missions, science in lunar orbit, and human-tended operations present substantial challenges for mission planning, operational preparedness, and astronaut training.

Prior research supports the use of extended reality (XR) in astronaut training and mission simulation, showing its promise in addressing similar challenges through immersive visualization and interaction for spacecraft pilot training in Lunar Orbital Platform-Gateway scenarios [3]. NASA's operational VR training systems have demonstrated that hardware-in-the-loop simulations are essential for astronaut certification, with crew required to complete VR training on critical systems before ISS missions [4]. Augmented reality has also shown promise, with concepts transferable from underwater training to space environments [5]. Recent studies further demonstrate VR's effectiveness as psychological countermeasures in isolated space environments [6] and NASA's operational use of AR and VR aboard the International Space Station [7], including for specific instrument upgrades [8]. These efforts show the value of immersive tools to enhance astronaut preparation and operational insights.

Barrios Technology, in partnership with an interdisciplinary student team from Purdue University’s Data Mine and Angelo State University, has developed the Gateway Visualization Tool (GVT). GVT is an interactive XR Platform designed to support mission planning, flight control visualization, astronaut training, and engineering analysis. By enabling intuitive visualization of complex systems and design interactions, GVT aims to significantly enhance the efficiency of NASA’s development process and enhance operational readiness. The following sections detail the tool’s development, its potential applications, and future directions of the GVT—demonstrating how visualization tools can enable the success and safety of humanity’s return to the lunar surface.

A. Background

To support NASA’s Artemis program, Barrios Technology collaborated with an interdisciplinary 15-student team from Purdue University’s Data Mine and Angelo State University to develop the Gateway Visualization Tool (GVT). The initial development phase focused on fleshing out a foundational framework for the GVT, emphasizing core functionalities with inherent adaptability. These foundational capabilities are designed for future expansion, enabling the tool to address specific niches as the Artemis program evolves. While the initial GVT version was developed using publicly available data, future enhancements will require a formalized partnership with NASA and access to NASA data.

This approach aligns with recent NASA initiatives that use virtual environments for operational planning, systems engineering, and training for engineering-focused applications for XR [4], including specialized astronaut training for lunar missions [9].

B. Objectives

The overarching development objective for the GVT was to create an immersive application in which the user could visualize NASA’s Gateway system from multiple perspectives and in various configurations, for mission planning, flight control, astronaut training, or engineering purposes. To guide this initial development, six primary objectives were established, the completion of which signified the successful first iteration of the GVT.

- 1) The tool being created must be capable of launching as a standalone desktop application.
- 2) The visibility of individual Gateway modules must be toggleable between shown and hidden states
- 3) The Gateway modules must be reconfigurable to create new docking arrangements.
- 4) The solar arrays and radiators must feature precise rotation control independently and with the option of unified rotation.
- 5) The Gateway must be visible from free-roam and multiple fixed camera perspectives.
- 6) The application must support initial XR functionalities, with a roadmap for enhanced VR integration.

In addition to these primary objectives, several secondary goals were achieved, further enhancing the tool’s effectiveness. The GVT presents the Gateway within a simulated space environment, including the Earth, Sun, and Moon, with the Gateway model following a nominal elliptical orbit around the Moon’s south pole. Users can save three distinct GVT states, preserving the configuration and visibility of all Gateway components and the current camera perspective for later reloading. The GVT also includes an integrated help menu explaining the user interface and features to facilitate user navigation and interaction. Finally, the design incorporates visual and audio cues to simplify user interaction and improve overall accessibility.

II. ENGINEERING DECISIONS AND RATIONALE

The development of the Gateway Visualization Tool (GVT) involved a structured series of engineering decisions shaped by technical constraints, team organization, and project objectives. Key design choices were guided by the dual aims of producing an immersive, interactive, and extensible visualization tool specifically for the Gateway, while also establishing a foundational framework adaptable for broader Artemis program visualization needs. This section outlines the primary constraints that influenced the development path and presents a comparative analysis of the development platforms considered.

A. Constraints

Early identification of project constraints was crucial for timely adaptation and avoidance of foreseeable obstacles. Key constraints included:

Hardware Equipment: Hardware resources, particularly VR equipment, were limited. A Meta Quest Two headset was available temporarily, facilitating intermittent VR testing. Consequently, research was conducted into VR simulation techniques not reliant on continuous headset availability and into best practices for VR integration to inform future development iterations.

Cross Teamwork Communication and Skills: The project team comprised students from Purdue University and Angelo State University, necessitating collaboration across both remote and in-person settings. This distributed structure required clear communication protocols and a shared platform for managing the Unreal Engine project and associated 3D assets. The interdisciplinary nature of the team also brought a diverse range of experience levels with Unreal Engine, from beginner to intermediate. This distribution amplified the need for effective team communication, collaborative learning, and adaptive problem-solving strategies.

Public Data: Development relied exclusively on publicly available data. This included limited 3D models of Gateway elements and visiting vehicles (e.g., Orion models) and orbital trajectory information [10], [11]. This constraint necessitated creative approaches to model implementation and a strong design emphasis on ensuring straightforward future integration of more comprehensive, proprietary Gateway data. Furthermore,

Game Engines	Unreal Engine - Ease of Integration With NASA - C++ / Blueprints - Highly Portable - Better Graphics	Unity - Beginner Friendly - C# / JavaScript - More Versatility	<i>visual enhancements</i> Blender - Better 3D modeling - Python / C++ - Steep Learning Curve
	Blueprints - Unreal Engine Native - Uses a node-based interface - Better suited for animations and UI design	C++ - Unreal Engine Native - More familiar for experienced coders - Easier to scale and read	Python - Native to Blender - More widely known software language - Manipulate objects, edit meshes, create custom UI

Fig. 1. Comparative Analysis Table

by compelling a focus on core, adaptable system functionalities rather than dataset-specific intricacies, this reliance on general public data inherently supported the strategic objective of creating a versatile foundational tool, primed for subsequent expansion and broader application within the Artemis program.

B. Comparative Analysis

Choosing the right software to use was crucial to the tool's development, as multiple software packages could have met the project's core functional requirements, with wide implications for the final product's compatibility with current NASA systems. Initial considerations involved combining specialized visual enhancement software with a separate modeling engine. However, the discovery of a readily available .GLB file on NASA's 3D resources Gateway Lunar Space Station page, containing a detailed and textured 3D model of the Gateway structure, largely obviated the immediate need for separate visual enhancement tools and shifted the focus towards integrated development environments [10].

C. Software Rationale

Unreal Engine emerged as a strong contender due to its comprehensive feature set and proven applications in NASA software ecosystems [12].

The engine's ability to render realistic space environments and handle complex 3D configurations [13], combined with established success in scientific visualization applications [14], enabled the project to meet its primary objectives while also providing the flexibility to incorporate additional features that enhance the GVT's overall versatility and potential for future expansion.

III. SIMULATED ENVIRONMENT

Unreal Engine offers a robust and flexible framework for creating immersive, interactive environments, making it an ideal choice for visualizing complex systems in a compelling way. For the development of GVT, it enabled the development of a dynamic space simulation that balances visual realism with representational clarity for mission-related contexts. Key components of the simulated environment include a realistic star field, accurately scaled celestial bodies (Earth, Moon, Sun), and a detailed model of the Gateway space station.

Interactive features and cinematic camera paths further enhance the user experience, supporting engagement and deeper understanding of spatial relationships in a lunar orbital context.

A. Stellar Backdrop

An immersive stellar backdrop was created by surrounding the simulation environment with a scaled version of the built-in sphere of Unreal Engine, enlarged to encompass the entire scene and serve as the star field container. The sphere's geometry was made invisible from the inside by flipping its normals, and a high-resolution, realistic star field texture was imported and applied to its surface. The texture and sphere were scaled relative to the Earth-Moon system to maintain a convincing sense of depth and astronomical scale. To further enhance the realism of the backdrop, post-processing effects such as bloom and exposure adjustment were implemented. A dynamic camera fly-through was also designed, resulting in an engaging and immersive experience within the GVT environment.

B. Celestial Bodies Setup

The Earth, Moon, and Sun are represented at accurate relative scales and initial orbital positions by applying a 1:1 scale where 1 Unreal Unit (UU) equals 10 kilometers. Based on this scale, the average Earth-Moon distance of 384,000 kilometers was represented by positioning the Moon 38,400 Unreal Units away from the Earth in the scene. This approach allowed us to maintain correct spatial proportions while keeping the simulation within a manageable size for the Unreal Engine environment. Realistic directional lighting simulating sunlight was implemented using Unreal Engine's built-in lighting system to create natural illumination. High-resolution textures were applied to both the Earth and Moon, enhancing surface detail and overall visual realism.



Fig. 2. Earth, Moon, and Sun Placement

C. Gateway Station Placement

The Gateway space station model was placed in a simplified representation of a Near-Rectilinear Halo Orbit (NRHO) around the Moon [11]. Although the model does not simulate true orbital motion, its position follows the intended Gateway orbital path to provide contextual accuracy regarding its intended operational environment. Interactive exploration and movement within the station were implemented using the Unreal Engine blueprint scripting system, allowing users to navigate the structure dynamically.

IV. SYSTEM ARCHITECTURE

The architecture of the GVT was designed with modularity, clarity, and future expansion in mind. This section outlines how the tool is organized across its core components: the breakdown of the modules, user interface systems, and extensibility features.

A. Modular Construction of the Gateway System

To add interactions within the virtual environment. The Gateway model had to be broken down into the whole of Gateway with the modules together, as well as separate each module with their respective classes. This is done using object-oriented programming principles.

Object-Oriented Structure: Upon importing the .glb Gateway model into Unreal Engine, the default behavior treated each module as a static mesh actor with no interdependencies or blueprint logic. These actors lacked component hierarchies and did not support interaction or runtime behavior.

To introduce the desired interactivity, each module was refactored as a child Blueprint Actor of a centralized Gateway parent class. This parent Blueprint functions as the root controller for all module interactions. Each child module—such as IHAB, Airlock etc.—inherits from Gateway, an abstract class that provides base functionality such as event dispatchers, module interfaces, and shared interaction logic.

The Gateway Parent Class contains:

- References to each module's Blueprint instance
- A UI controller component that binds to widget blueprints for user interaction.
- Centralized event handling logic, such as input dispatching, camera shifts, and visibility toggles.
- Utility functions to facilitate user input not supported by Unreal defaults, such as clicking on modules.

Each Child Module contains:

- A static mesh component representing the geometry.
- Attributes (e.g., "Moveable") to ensure correct interactions.
- Bounding boxes and functions used for dot product calculations are used to show only available docking ports.

This architecture ensures that interactions such as hovering, clicking, and moving modules are handled consistently across all components of Gateway.

Modularity of Assets: Each Gateway module was treated as an independent and child blueprint actor class within the simulated environment. This still allowed the different modules to be individually manipulated, allowing the users to have full control of Gateway and what modules they want to move around and what functionality they want to access. This allows full dynamic access when the user wants to access Gateway as a whole or an individual module.

B. UX and Interaction System

The UX was developed with a mission-first approach, prioritizing ease of use during simulation training, clarity of information for mission planning, and access to technical tools for engineering. These tools help with training, mission planning, and engineering analysis. The UX is successfully developed using the Unreal Engine system, enabling modular widget design. The tool is not simply a visual representation, but is an interactive simulation environment that enables users to perform specific tasks with precision and clarity. This is possible by tailoring the interface to support the different workflows of training, mission planning, and analysis.

C. Extensibility and Scalability

The GVT's extensibility has been established with considerations in three different categories to support NASA's Artemis mission.

Astronaut training: GVT can help astronauts-in-training build spatial awareness and practice basic procedures [3] by allowing them to freely explore in a safe, repeatable Gateway environment. Included interactive elements like labeled components, rotational sliders, and contextual tool tips aid navigation without prior experience, supporting a self-guided and immersive training process. These and future GVT features will help ease training by promoting muscle memory and procedural understanding. This approach aligns with established NASA VR training protocols, where astronauts must complete VR certification on critical systems before ISS missions [3].

Mission training: Users of GVT can test hardware layouts and simulate workflows. Features included in the module, such as saving modules and reloading them at a later time helps planners visualize and refine configuration before physical implementation. GVT also lays the foundation for future versions to provide evaluation of astronaut movement.

Engineering analysis: GVT enables evaluation of the technical feasibility of module designs by identifying potential part collisions and visually confirming that configurations meet basic design constraints. This is achieved by integrating a clear workflow for uploading module designs to GVT, utilities for module loading, structural overlays, and highlighting critical areas (connection points between modules).

V. TOOL CAPABILITIES

The GVT incorporates several key features, developed in consultation with Barrios Technology, to establish its foundational capabilities. These features are intended to support astronaut training, aid mission planning, and provide an interactive

model for exploring the Gateway Space Station. Core functionalities include: module visibility control (hide/show), module reconfiguration, configuration saving, solar panel and radiator rotation, multiple camera views, and initial VR integration. A dynamic orbit simulation is among the features currently under development. The following subsections describe each of these capabilities in more detail.

A. Hide/Show Modules

Users can toggle the visibility of individual Gateway modules and two attached robotic arms using designated numeric keys (1-7). This functionality enables visualization of the Gateway assembly sequence, representing progressive stages such as its construction in orbit or the occurrence of visiting vehicles. Additionally, this feature can be used to temporarily toggle modules to enable visibility of the underlying content. This feature was implemented using Boolean toggles corresponding to actor reference variables pointing to each module set at application startup.

B. Move Modules

A key capability of the GVT is the visualization of module reconfiguration, simulating preparations for the arrival and docking of visiting vehicles or new hardware components. In the interest of creating a foundational tool, this project set its scope on reconfiguring the existing Gateway modules, with an emphasis on dynamic configurations over preloaded setups. All modules that can be reconfigured are part of the "Move" class. This design allows for the straightforward integration of new models, such as visiting vehicles, into the existing movement system in future iterations by assigning them to this class.

This feature is implemented through the use of a UI button-activated function that returns the pointer of a module when clicked. This pointer is used as the object of functions that save the three-dimensional location and rotation of the clicked module. By placing static boxes at each port of every Gateway module, the translations of the modules can be mapped onto those of the clicked boxes. This mapping allows for correct placement of module origins when moving, ensuring each Gateway module orientation remains consistent.

C. Save Module Configurations

The GVT includes three preset module configurations of Gateway—I-Form, T-Form, and L-Form (displayed in figure 3). These configurations resemble different orientations of Gateway, serving as "default" positions within the tool. These presets provide consistency and a baseline for all users.

In addition, users can dynamically save and load custom configurations created using the tool's various modification features. Save files are created when the user clicks the save button in the bottom right of the UI, and selects a file to write to. These custom configurations can be reloaded in subsequent sessions, even after the application has been closed and restarted.

This is a signature aspect of the tool's current simulation abilities as it facilitates engineering decision-making on the

part of the user. Having the ability to create and compare multiple mission configurations, visiting vehicle docking locations, and solar array rotations fulfills the simulation tool's purpose of informing mission design.



Fig. 3. Preset Module Configurations

D. Solar panels and Radiator Rotations

The GVT allows manual, degree-precise control over the rotation of solar panels and radiators. Users can manipulate these components collectively or individually via UI elements or keyboard hotkeys.

Slide Bar: A UI slide bar enables users to drag a control from 0° (left) to 360° (right), providing a quick method for rotating all solar panels and radiators in unison. The slide bar is made inside the widget User Interface Blueprint. Within the blueprint, there are events that allows for actions to take place when the user clicks and holds down the mouse button, drags along the bar, and releases the mouse button. With this, a function can be coded with blueprints using a "set actor rotation" node. When the user moves the bar to the right, it will increase by 1° to the right until hitting 360°. And when the user drags the bar to the left, the rotation value decreases by 1° until hitting 0°.

Precise Rotation: The user is able to move each solar array and radiator individually or all together by typing into the input field that is located at the top right corner. To rotate all the components together, after typing a rotation degree, the user can press the up arrow key to move all the components in one direction and the down arrow key to reverse the direction. Additionally, when the arrow keys are pressed, the user can repeatedly press or hold down the keys to automatically rotate the components without repeatedly pressing the keys.

For individual rotation, hotkeys allow the user to choose which solar array or radiator to rotate manually. 'I' rotates a specific array, and 'K' reverses its rotation. Similarly, 'H' and 'N' control a panel on the iHab module, while 'J' and 'M' control a panel on the airlock module. These functions are implemented within the Gateway's actor Blueprint, taking user key presses as input, updating a float rotation variable, and applying the rotation to the selected module component.

E. Camera Views

The GVT includes three predefined orthographic camera perspectives—Right, Left, and Top—that support the spatial

analysis of Gateway configurations. Users may access each view via labeled UI buttons: camera. This feature will aid in the future user's ability to simulate docking scenarios

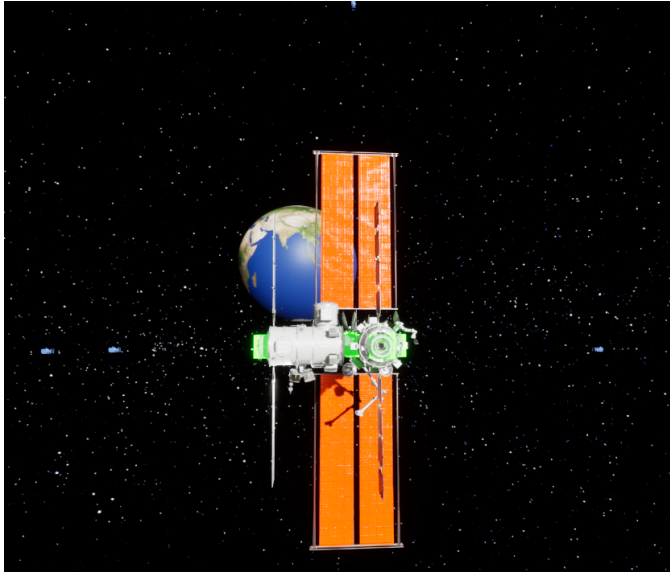


Fig. 4. Camera Actor Placements

The functionality is implemented using dedicated camera actors. Each actor is positioned orthogonally relative to the Gateway. The camera actors are hidden from the environment (visibility disabled) and remain inactive until triggered. Users may select one of four UI buttons labeled “Top”, “Right”, “Left”, and “Original” to change their perspective. The former three buttons result in the related camera actor's view to transition onto the screen. The latter button returns the user to their current position within the environment. All view transitions occur within real-time. The camera transition is interpolated to ensure a smooth shift between viewpoints, avoiding abrupt changes. The preset views provide users with control over spatial perspective, enhancing usability and situational awareness during configuration tasks

F. Orbital Mechanics

A low-fidelity simulation of Gateway's orbit was developed using an elliptical mathematical model within Unreal Engine, building upon established principles of lunar trajectory design [11]. The orbiting visualization, triggered by user key press ('O'), transitions the Gateway model from a static representation to a dynamic, looping motion that reflects the simplified orbital path. This approach enables users to gain a conceptual understanding of Gateway's position and movement over time.

Continuous orbital simulation loops were implemented in addition to the features included in the tool previously. This allowed for orbit simulation without interrupting other capabilities, such as camera movement or UI-based module control. While simplified, this capability effectively demonstrates Gateway's conceptual orbital behavior.

All orbital behavior in this GVT version is derived from publicly available datasets and does not yet represent the unique Near-Rectilinear Halo Orbit (NRHO) planned for Gateway that offers continuous line-of-sight with Earth and minimal fuel requirements for lunar access. Future iterations, with access to precise orbital parameters and NASA telemetry, aim to incorporate true-to-life NRHO modeling. This would significantly enhance the tool's utility for visualizing mission-critical orbital scenarios, aiding engineers, mission planners, and astronauts.

G. Virtual Reality Integration

The current GVT iteration offers initial VR support compatible with industry-standard VR hardware requirements [15]. Using a Meta Quest 2 headset, Unreal Engine's built-in VR preview capabilities allow the simulation environment to be launched directly within the headset. Setup involves creating a Meta Quest account, installing the Meta Quest Link application, and connecting the headset to the development computer.

Once launched in VR, the user can view the Gateway model within the immersive stellar environment, with the Moon, Earth, and UI elements visible. Head tracking enables free look-around capabilities, providing enhanced spatial awareness of the station's configuration and its position within the lunar environment. This foundation builds upon NASA's proven VR training methodologies, where hardware-in-the-loop simulations have been essential for astronaut certification [3].

Simulated VR integration can be made possible by Unreal Engine 5's virtual reality mode in addition to available add-ons. Rather than requiring a headset to experience the program's VR capabilities, simulating the headset would ease the hardware requirement for this aspect of the program, since any user could view the program in VR mode, even if a full setup is not readily available. In this mode, the user would be operating the program in an ocular view (i.e. left- and right-eye cameras separated) with different keyboard and mouse inputs functioning as the VR controllers and adjusting the camera position. Ultimately, a simulated VR mode is a mechanism that could reduce the barrier to entry for users without a headset seeking to fully utilize the program and its immersive capabilities.

VI. USER INTERFACE AND DESIGN RATIONALE

The User Interface (UI) of the GVT was designed to be intuitive, improving user immersion and operational efficiency. Users engage with the interface through strategic button placement to minimize screen obstruction, clear labeling to guide user actions, and contextual help features to reduce learning time. Additionally, the button features are clearly labeled to indicate their features, and keyboard hotkeys support rapid access to frequent functions, while sub-panels and tutorials maintain a clean, informative interface. Users can also freely navigate the simulator using standard mouse input, and features like the rotational slider provide intuitive, button-free control over module orientation. These interaction mechanisms

ensure that users can perform their tasks efficiently and with confidence, whether training, planning, or analyzing.

A. Button Placement

Interactive UI buttons are primarily placed in the corners of the screen to minimize obstruction of the main viewing area. For features that require multiple options, such as camera selection or save/load functions, a primary button reveals a contextual sub-panel, maintaining an uncluttered interface while providing clear guidance.

In-Simulation Helper Button: A readily identifiable help button (indicated by a question mark icon) is positioned in the top right corner. When activated, this button displays contextual information and instructions for each of the GVT's features, supporting user comprehension and reducing learning time.

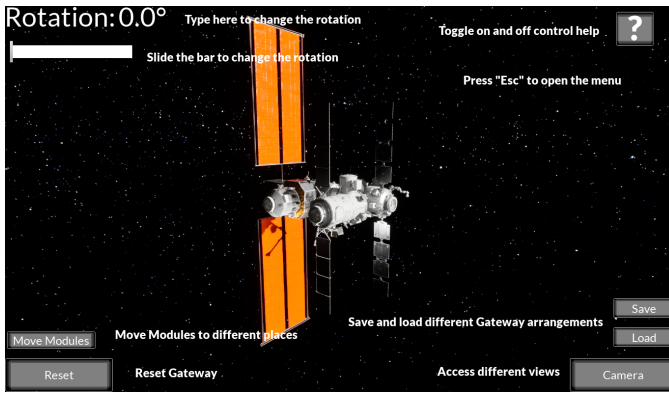


Fig. 5. Tutorial Text

Button Signifier Labeling: An important feature for users to understand what actions they can perform inside the tool is ensuring that the buttons are clearly labeled to indicate their corresponding actions. These labels serve as identifiable signifier that allow users to anticipate the outcome of their interactions with the interface.

Non-Screen Button Features: To maximize the field of view and provide rapid access to frequently used functions, several features are controllable using keyboard hotkeys. These include module visibility toggles and certain solar panel/radiator rotation control. An on-screen tutorial image, accessible via the Help system, details these hotkeys and button functionalities to support user learning and efficient operation.

Rotational Slider: The Rotation section, displayed in the top-left corner of Figure 5, allows users to control the rotational orientation of the module. This section is not a traditional button but provides users with fine-grained control over the module's orientation, making it easier to position components accurately during simulation.

B. Future Goals

Although the current GVT iteration provides valuable foundational capabilities, several key areas for future development have been identified to enhance its utility, realism, and scope.

Improved Orbital Modeling: A primary goal is to transition from the current simplified orbital simulation to an accurate Near-Rectilinear Halo Orbit (NRHO) model [11]. This would involve integrating NASA's precise orbital data to provide users with accurate orbital dynamics and maneuver simulation capabilities.

Better VR Integration for EVA Simulations: Further development of VR functions aims to support immersive extra-vehicular activity (EVA) training scenarios that meet established VR system requirements [16], building upon NASA's existing VR training infrastructure [3], [17]. Similar VR approaches have already proven effective for spacecraft piloting training in lunar orbital scenarios [9], demonstrating the feasibility of comprehensive Gateway operations training that could extend from basic navigation to complex docking simulations and mission rehearsal. Because of its proven effectiveness, a main goal is to ensure a smooth and cohesive user experience, primarily by doing things like optimizing frame rate to minimize the effect of motion sickness in long training sessions, and ensuring the program is compatible with all major VR headsets.

Docking Simulation Capabilities: Future iterations plan to incorporate the simulation of docking sequences for visiting spacecraft, enabling users to visualize and analyze these critical operations under realistic orbital conditions.

Lunar Base Interaction Visualization: Expanding the tool to include visualizations of lunar surface assets and their interaction with the Gateway (e.g., communication links, crew transfers, payload logistics) is crucial for systems engineering and integrated mission planning. The potential of Metaverse concepts for planetary exploration, such as for Mars, further supports this direction [18].

Interior Gateway and System Visualization: Expanding GVT into the internal layout of the Gateway would open possibilities for crew habitat familiarization, internal systems walkthroughs, and operational procedure training, modeled after International Space Station (ISS) analogs.

VII. SYSTEM EVALUATION

The GVT was evaluated based on real-time performance, hardware requirements, and scalability across different system configurations. The tool was developed using Unreal Engine 5.5, implementing Blueprint actors and Widget Blueprints for user interface components.

A. Development Hardware Configuration

Two primary development systems were utilized to ensure performance validation with specifications listed in Table I.

B. Performance Analysis

Development Performance: Environment creation and real-time rendering performed smoothly on both systems.

TABLE I
DEVELOPMENT HARDWARE CONFIGURATION

Component	Primary System	Secondary System
Purpose	Environment Creation	Integration Testing
Processor	Intel Core i7	AMD Ryzen 9 6900HS
Graphics	NVIDIA GeForce RTX 3060	AMD Radeon (integrated)
Memory	–	16 GB RAM
Display	–	165 Hz refresh rate
Model	–	ASUS ROG Flow X16
Operating System	Windows 11	Windows 10
Performance	Stable real-time rendering	60–90 FPS at high settings

Primary performance bottlenecks included texture import processing and shader compilation times, consistent with typical UE5 development workflows. Once assets were compiled, both systems maintained stable performance during design iteration and testing phases.

Runtime Performance: The GVT achieved consistent frame rates of 60-90 FPS on the secondary development system when configured at high graphical settings (scale: low, medium, high, epic). This performance was maintained during Gateway simulation and environment interaction scenarios.

Virtual Reality Performance: VR testing utilized a Meta Quest 2 headset in wireless mode. The system achieved:

- **Frame Rate:** Consistent 60 FPS average
- **Tracking:** Responsive head tracking with minimal latency
- **Visual Quality:** High fidelity maintained throughout testing
- **Stability:** No significant performance degradation during extended sessions

The successful VR performance on Quest 2 hardware demonstrates accessibility potential for deployment, meeting minimum VR hardware requirements [15] at an accessible price point. Future optimization work can investigate compatibility with ultra-budget VR solutions, including Google Cardboard through reduced graphical settings and mobile application packaging.

C. System Requirements and Recommendations

Recommended specifications were identified based on established VR compatibility standards [15], [16] and can be found in Table II.

D. Scalability Assessment

Compatibility testing confirmed the GVT's scalability across hardware configurations. Desktop builds successfully executed on systems below VR minimum requirements when operated in standard display mode, including integrated graphics configurations. This scalability enables deployment across

TABLE II
SYSTEM REQUIREMENTS FOR END USERS

Component	Minimum	Recommended
<i>Core Requirements</i>		
Operating System	Windows 10 64-bit or 11	Windows 10 64-bit or 11
Processor	Intel i3-4130 / AMD FX 4300	Intel i7 / AMD Ryzen 7
Graphics	GTX 750 Ti / RX 460	RTX 3060 / RX 6600+
Memory	4 GB RAM	16 GB RAM
Storage	2 GB available	2 GB available
Display	1920×1080	1920×1080
<i>VR-Specific Requirements</i>		
VR Processor	Intel i5-4590 / FX 8350	Intel i7 / AMD Ryzen 7
VR Graphics	GTX 1060 / RX 480	RTX 3060 / RX 6600+
VR Memory	8 GB RAM	16 GB RAM
Connectivity	USB 3.0, HDMI 1.3+ or DP 1.2+	USB 3.0, HDMI 1.3+ or DP 1.2+
VR Headset	Meta Quest 2	Meta Quest 2/3, HTC Vive, or compatible OpenXR device

diverse institutional hardware environments while maintaining core functionality for users without VR-capable systems.

E. User Feedback

The GVT was presented to stakeholders at Barrios on a weekly basis and NASA employees involved in the Gateway program following the completion of the software build described in this paper. As such, end user feedback was incorporated regularly and informed the design of the tool at every stage, as well as the future directions outlined in this paper.

Incorporated Stakeholder Feedback: A large portion of stakeholder feedback involved making the tool intuitive to users while maximizing flexibility and versatility in mission planning and engineering decision making. For this reason, dynamic solutions to the tool's requirements were chosen. One such case of this was in fulfilling the requirement of having three preset configurations. Instead of developing a static configuration method, the "Move Modules" and "Save/Load" features were created to fulfill the requirement while allowing the user to configure their own arrangements in the process of mission planning.

End User Feedback: In concordance with recommendations made by the GVT stakeholders, the end users were satisfied with the configurability of the tool. During the presentation out-brief, one participant stated: "The catch-all for configurations is great!" Another member of the group noted: "A help menu overlay during simulation would help clarify controls not on screen." For this reason, a static tutorial button was placed in the top right to explain each feature of the tool to users. End user feedback affirmed overarching development

choices while giving insights on how to make the tool more accessible to new users.

VIII. CONCLUSIONS

The Gateway Visualization Tool, developed by an interdisciplinary student team over a 16-week period in Spring 2025, represents a foundational step towards creating a comprehensive suite of visualization tools for NASA's Artemis program. This initial iteration successfully established core functionalities, including module manipulation, environmental simulation, and preliminary VR integration. These features lay the groundwork for a versatile tool capable of supporting mission planning, engineering analysis, and astronaut training related to the Gateway and lunar surface operations.

GVT's ongoing development will be guided by continued input from stakeholders, including Barrios Technology and NASA. As the GVT evolves, its potential to serve as a valuable asset for training, mission planning, and engineering decision support for the Artemis program will continue to grow. Beyond its direct applications, the GVT also underscores the transformative impact specialized visualization solutions can offer, paving the way for a new generation of tools crucial for navigating the complexities of future Artemis missions.

REFERENCES

- [1] NASA, "Artemis plan: Nasa's lunar exploration program overview," September 2020.
- [2] NASA, "About gateway," 2024.
- [3] E. P. A. D. Garcia, J. Schlueter, "Training astronauts using hardware-in-the-loop simulations and virtual reality," in *AIAA Scitech 2020 Forum*, (Orlando, FL), American Institute of Aeronautics and Astronautics, January 2020.
- [4] E. Paddock, "Nasa engineering vr overview," technical report, NASA, July 2023.
- [5] B. Nuernberger, R. Tapella, S.-H. Berndt, S. Y. Kim, and S. Samochina, "Under water to outer space: Augmented reality for astronauts and beyond," *IEEE Computer Graphics and Applications*, vol. 40, no. 1, pp. 6–14, 2020.
- [6] J. Sharp, J. Kelson, D. South, A. Saliba, and M. A. Kabir, "Virtual reality and artificial intelligence as psychological countermeasures in space and other isolated and confined environments: A scoping review," *Acta Astronautica*, vol. 232, pp. 666–677, 2025.
- [7] A. M. Braly, "Augmented reality improves procedural work on an international space station science instrument," *Human Factors*, vol. 61, pp. 866–878, January 2019.
- [8] J. Kellogg, K. Andrea-Liner, and J. Jennings, "Augmented reality assisted astronaut operations in space to upgrade the cold atom lab instrument," in *Proceedings of SPIE*, vol. 12447, 2023.
- [9] S. R. R. E. M. B. Bruguera, V. Ilk, "Use of virtual reality for astronaut training in future space missions - spacecraft piloting for the lunar orbital platform -gateway (lop-g)," in *Proceedings of the 70th International Astronautical Congress*, (Washington, DC), October 2019.
- [10] NASA, "Gateway lunar space station," August 2024. Latest Update: 2025.
- [11] D. C. Davis, B. P. McCarthy, and E. M. Zimovan-Spreen, "Perturbations and recovery in the gateway near rectilinear halo orbit," in *Proceedings of the 2023 AAS/AIAA Spaceflight Mechanics Meeting*, no. AAS 23-190, (Austin, TX, United States), American Astronautical Society and American Institute of Aeronautics and Astronautics, January 2023. NASA Document ID: 20220019129. Meeting held January 15-19, 2023.
- [12] R. Barry, "How nasa is using virtual reality to prepare for science on moon," tech. rep., NASA Science Mission Directorate, Game Changing Development Program, 2023.
- [13] Epic Games, "Hardware and software specifications for unreal engine," 2024.
- [14] K. I. Friese, M. Herrlich, and F. E. Wolter, "Using game engines for visualization in scientific applications," in *New Frontiers for Entertainment Computing*, pp. 11–22, Springer, 2008.
- [15] Meta, "Meta quest link compatibility and requirements," 2024.
- [16] B. Lang, "Check if your pc is vr ready for oculus, htc vive, valve index, wmr headsets," June 2024.
- [17] NASA, "Virtual reality training lab," 2023.
- [18] C. D. Tate and A. M. Annex, "The metaverse is here—so let's use it to explore mars," in *Proceedings of the Lunar and Planetary Science Conference*, 2023.

ACKNOWLEDGMENT

We want to thank our mentors Jacob Wade, Jeanette Snyder, Dr. Erdoğan Doğdu, Dr. Mark Daniel Ward, Laney Ciaccio, Devin Vyain, Ginger Kerrick Davis, Yazan Mahmoud, Pete Dragnev, The Data Mine Team, partners at Angelo State University, and Barrios for their assistance with this project.